Proyecto Final Bases de Datos

REVIEWS DE AMAZON



Pilar Pérez Hernández, María Roca Ramón

ICAI | UNIVERSIDAD PONTIFICIA COMILLAS

Pilar Pérez Hernández
María Roca Ramón
(2ºB iMAT)

1. INTRODUCCIÓN

El presente documento constituye la memoria del proyecto de la asignatura Bases de Datos, correspondiente al curso 2025-2026. El proyecto ha consistido en el desarrollo de una aplicación Python que integra tres sistemas de bases de datos distintos: MySQL (base de datos relacional), MongoDB (base de datos documental) y Neo4J (base de datos orientada a grafos), para el almacenamiento, consulta y visualización de datos de reviews de productos de Amazon.

Los conjuntos de datos utilizados provienen del repositorio público Reviews de Amazon y corresponden a las siguientes categorías de productos:

Categoría	Fichero	Nº de reviews
Toys and Games	Toys_and_Games_5.json	167.597
Video Games	Video_Games_5.json	231.780
Digital Music	Digital_Music_5.json	64.706
Musical Instruments	Musical_Instruments_5.json	10.261

Cada review sigue un formato JSON con los siguientes campos: identificador del revisor (`reviewerID`), identificador del artículo (`asin`), nombre del revisor (`reviewerName`), votos de utilidad (`helpful`), texto de la review (`reviewText`), nota (`overall`), resumen (`summary`), timestamp Unix (`unixReviewTime`) y fecha legible (`reviewTime`).

1.1. OBJETIVOS DEL PROYECTO

El proyecto está formado por cinco partes obligatorias y dos opcionales:

- **Diseño y carga de datos:** Repartir la información entre MySQL y MongoDB con una justificación clara, y cargar los cuatro ficheros JSON leyendo línea a línea (sin cargar todo en memoria).
- **Visualización de datos:** Un menú interactivo en Python con varias opciones de visualización (histogramas, curvas de popularidad, nubes de palabras, etc.) que consulte ambas bases de datos.
- **Aplicación con Neo4J:** Calcular similitudes entre usuarios con la correlación de Pearson, crear grafos de relaciones en Neo4J y analizar artículos populares con artículos en común.
- **Inserción de un nuevo dataset:** Añadir un quinto dataset (Office Products) a las bases de datos ya existentes, teniendo en cuenta que un usuario puede tener reviews en varias categorías.
- **Tableau y Machine Learning:** Exportar datos para Tableau, describir un modelo de recomendación y devolver los 10 artículos más populares no consumidos por un usuario.
- **Implementación de modelo de Machine Learning (Opcional):** Implementar el modelo de filtrado colaborativo descrito en la parte anterior, con división en conjuntos de entrenamiento y test, evaluación con métricas RMSE y MAE, y demostración de recomendaciones.
- **Mejora en visualización – Dashboard interactivo (Opcional):** Desarrollar un dashboard web interactivo son Streamlit que sustituya el menú por terminal, ofreciendo una interfaz más intuitiva con gráficas interactivas generadas con Plotly.

1.2. ESTRUCTURA DEL PROYECTO

El proyecto está compuesto de 8 módulos Python y un fichero de dependencias con un total de 2.875 líneas de código y 59 funciones.

Fichero	Descripción	Líneas	Funciones
`configuracion.py`	Credenciales y rutas de datos	35	0
`load_data.py`	Carga de datos en MySQL y MongoDB	271	10
`menu_visualizacion.py`	Menú visualización y recomendación	609	13
`Neo4Jproyecto.py`	Aplicación con Neo4J	608	15
`inserta_dataset.py`	Inserción de nuevo dataset	186	4
`exportar_datos_tableau.py`	Exportación de datos a Excel	66	1
`modelo_ml.py`	Modelo de Machine Learning (opcional)	538	12
`dashboard_streamlit.py`	Dashboard interactivo streamlit (opcional)	554	4
`Requirements.txt`	Dependencias del dashboard	8	-
TOTAL		**2.875**	**59**

1.3. TECNOLOGÍAS Y LIBRERÍAS UTILIZADAS

El proyecto se ha desarrollado en Python 3 y se ha hecho uso de las siguientes tecnologías y librerías:

Tecnología / librería	Uso en el proyecto
MySQL + PyMySQL	Almacenamiento de tatos estructurados (reviewers, products, reviews)
MongoDB + PyMongo	Almacenamiento de datos textuales (reviewText, summary)
Neo4J + neo4j (driver)	Modelado de grafos: similitudes, relaciones usuario artículo
matplotlib + seaborn	Generación de gráficas de visualización
WordCloud	Generación de nubes de palabras
openpyxl	Exportación de datos a formato Excel (.xlsx) para Tableau
json	Lectura y parseo de los ficheros JSON línea a línea
Csv	Escritura del fichero auxiliar de similitudes de Pearson
datetime	Parseo de fechas del formato del JSON a tipo DATE de MySQL
Streamlit	Framework web para el dashboard interactivo
Ploty	Gráficas interactivas en el dashboard (zoom, hover, descarga PNG)
Pandas	Procesamiento de datos tabulares y tablas interactivas en el dashboard
Random	Selección aleatoria de artículos (Neo4J) y división train/test (ML)

Pilar Pérez Hernández
María Roca Ramón
(2ºB iMAT)

scikit-learn / math	Cálculos estadísticos del modelo de filtrado colaborativo

En las siguiente secciones se detallará el proceso de desarrollo de cada una de las partes del proyecto, describiendo las decisiones de diseño tomadas, los desafíos encontrados y las soluciones adoptadas.

Pilar Pérez Hernández
María Roca Ramón
(2ºB iMAT)

2. PRIMERA PARTE: DISEÑO Y CARGA DE DATOS

En esta sección explicamos cómo hemos diseñado la base de datos, por qué hemos repartido la información de esta forma entre MySQL y MongoDB, y cómo funciona el proceso de carga de los JSON.

2.1. JUSTIFICACIÓN DEL REPARTO MYSQL / MONGODB

Para decidir qué iba a cada base de datos, nos fijamos en la naturaleza de cada campo del JSON y en para qué es bueno cada sistema:

MySQL lo hemos utilizado para almacenar los **datos estructurados** que requieren: integridad referencial, consultas con JOINs, agrupaciones y ordenaciones eficientes. Son datos con un esquema fijo y con tipos de datos bien definidos (números, fechas, identificadores). En MySQL almacenamos:

- **Información de los revisores** (`reviewerID`, `reviewerName`): datos de identidad que se repiten en múltiples reviews y que nos conviene normalizar para evitar redundancia.
- **Información de los productos** (`asin`, `category`): el identificador único del producto junto con su categoría. El campo category no existía en el JSON original y se ha añadido para identificar de qué fichero proviene cada producto.
- **Datos cuantitativos de las reviews** (`overall`, `helpful_votes`, `total_votes`, `reviewDate`, `unixReviewTime`): notas, votos de utilidad, y fechas parseadas a tipo DATE de MySQL. Estos datos son los más consultados en las visualizaciones (agrupaciones por año, por nota, cálculos de popularidad, etc.).

MongoDB lo usamos almacenar los **datos textuales** y **no estructurados** de las reviews. Es especialmente adecuado para este tipo de datos porque nos permite almacenar documentos con campos de texto de longitud variable sin necesidad de definir un tamaño máximo de columna, y porque las consultas textuales (como la generación de las nubes de palabras) se realizan de forma más natural sobre documentos. En MongoDB se almacenan:

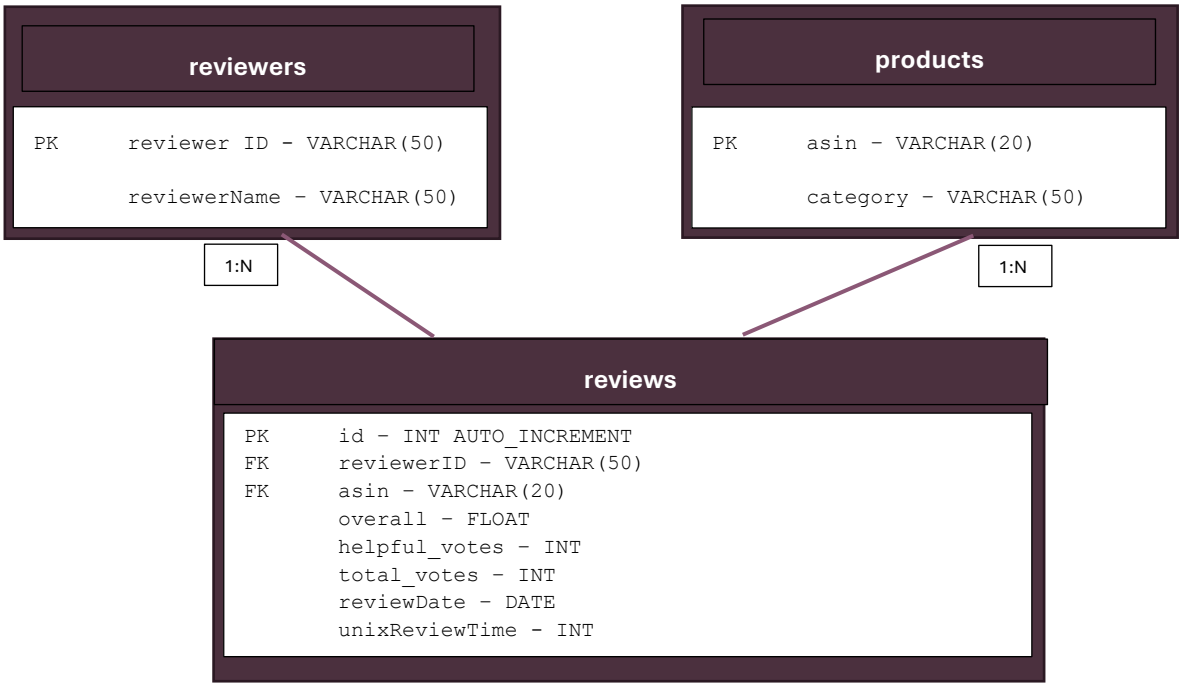
- **Texto de la review** (`reviewText`): el cuerpo completo de la review, que puede ser muy largo y variable en su longitud.
- **Resumen de la review** (`summary`): el texto resumido de la review. Este campo lo usaremos para generar las nubes de palabras
- **Campos de referencia** (`reviewerID`, `asin`, `overall`): se almacenan también en MongoDB para poder vincular cada documento textual con su revisor y producto correspondiente sin necesidad de hacer consultas cruzadas entre ambas bases de datos para operaciones sencillas.

Esta tabla resume el reparto de campos entre ambos sistemas:

Campo del JSON	MySQL	MongoDB	Justificación
`reviewerID`	■	■	Clave primaria en MySQL, referencia en MongoDB
`asin`	■	■	Clave primaria en MySQL, referencia en MongoDB
`reviewerName`	■		Dato estructurado, normalizado en tabla reviewers
`helpful`	■		Desglosado en helpful_votes y total_votes (enteros)
`overall`	■	■	Dato numérico para consultas SQL y filtrado en Mongo
`reviewText`		■	Texto largo y variable, ideal para almacén documental
`Summary`		■	Texto corto usado para nubes de palabras
`unixReviewTime`	■		Timestamp numérico para consultas temporales
`reviewTime`	■		Parseado a tipo DATE de MySQL (reviewDate)

2.2. ESQUEMA RELACIONAL DE MYSQL

El esquema relacional de MySQL está compuesto por tres tablas relacionadas entre ellas mediante claves primarias y foráneas. Se ha optado por un diseño normalizado para evitar la redundancia de datos y garantizar la integridad referencial:



2.2.1. TABLA `REVIEWERS`

Almacena los **revisores**. La clave primaria es `reviewerID`. Como un mismo usuario puede aparecer en varios ficheros (por ejemplo, alguien que ha revisado tanto videojuegos como juguetes), esta tabla normaliza sus datos y evita duplicados. El campo `reviewerName` admite nulos porque hay reviews que no lo incluyen.

2.2.2. TABLA `PRODUCTS`

Almacena los **productos** con `asin` como clave primaria. Añadimos el campo category para saber de qué fichero viene cada producto (Toys_and_Games, Video_Games, etc.), lo que nos permite filtrar por categoría en las visualizaciones.

2.2.3. TABLA `REVIEWS`

Es la **tabla principal**. Tiene un id autoincrementable como clave primaria (lo añadimos para identificar cada review de forma única) y dos claves foráneas hacia reviewers y products. El campo `helpful` del JSON, que es un array de dos elementos, lo desglosamos en `helpful_votes` y `total_votes`. La fecha `reviewTime` (que viene en formato texto «MM DD, YYYY») la parseamos a tipo `DATE` de MySQL. También conservamos `unixReviewTime` como entero.

Las relaciones son **1:N**: un revisor puede tener muchas reviews y un producto puede tener muchas reviews. Las claves foráneas garantizan que no pueda existir una review sin su revisor o producto correspondiente.

2.2.4 COLECCIÓN DE MONGODB: `REVIEWS`

En MongoDB usamos una única colección llamada `reviews` dentro de la base de datos `reviews_amazon`. Cada documento de la colección tiene la siguiente estructura:

```
{
      "reviewerID": "A2HD75EMZR8QLN",
      "asin": "0700099867",
      "reviewText": "texto largo de la review...",
      "summary": "resumen breve",
      "overall": 5.0
}
```

Los campos `reviewerID` y `asin` sirven de referencia para poder vincular los datos textuales con la información estructurada de MySQL cuando sea necesario (por ejemplo para la nube de palabras se obtienen los `asin` de una categoría desde MySQL y luego se consultan los `summary` correspondientes en MongoDB).

2.3. PROCESO DE CARGA DE DATOS

La carga de datos se realiza también mediante Python a través del fichero `load_data.py`.

Al ejecutarlo se lleva a cabo el siguiente proceso:

Paso 1: Se borran las bases de datos anteriores (si existen) y se crean de cero tanto en MySQL como en MongoDB, para partir siempre de un estado limpio.

Paso 2: Se crean las tres tablas en MySQL con sus claves y tipos de datos.

Paso 3: Se recorren los cuatro JSON de forma secuencial. Cada fichero se lee línea a línea con `for linea in fichero` y cada línea se parsea con `json.loads()`. Así, aunque un fichero pesase 40 GB, el consumo de memoria se mantendría constante.

Paso 4: Para cada review, se insertan los datos en MySQL: primero el reviewer (si no existe ya), luego el producto (si no existe), y por último la review con la fecha parseada.

Paso 5: En paralelo, se inserta un documento en MongoDB con los campos textuales y los de referencia.

Paso 6: Se hace commit en MySQL cada 10.000 reviews para no acumular demasiadas operaciones en una sola transacción. MongoDB no necesita commits explícitos porque cada `insert_one()` es atómico.

2.3.1. DESAFÍOS ABORDADOS EN LA CARGA DE DATOS

Lectura eficiente sin cargar en memoria. El enunciado prohíbe leer el fichero completo de una sola vez, usar pandas o `LOAD DATA`. La solución que hemos tomado es leer línea a línea con `for linea in fichero` (iterador lazy de Python) y parsear cada línea individualmente con `json.loads()`, manteniendo en memoria un único diccionario de review en todo momento.

Usuarios compartidos entre datasets. Un mismo usuario puede tener reviews en varios ficheros. Utilizamos un `set` en memoria (`reviewers_insertados`) para comprobar si el reviewer ya fue insertado antes de ejecutar el `INSERT`, y capturamos `IntegrityError` como segunda barrera de seguridad. El mismo mecanismo se aplica a los productos.

Parseo de fechas. El campo `reviewTime` tiene formato no estándar ("MM DD, YYYY"). La función `parsear_fecha()` lo convierte `a datetime.date` mediante `strptime()` con el formato `"%m %d, %Y"`, compatible directamente con el tipo `DATE` de MySQL.

Rendimiento de la inserción. Con casi medio millón de reviews, hacer commit por cada inserción sería inviable. Agrupamos las inserciones haciendo commit cada 10.000 reviews, equilibrando rendimiento y seguridad ante fallos.

2.3.2. FICHERO DE CONFIGURACIÓN

Toda la configuración (credenciales, nombres de bases de datos, rutas de ficheros) está centralizada en `configuracion.py`, para que los profesores puedan cambiar las rutas y credenciales desde un único sitio sin tocar el resto del código.

3. SEGUNDA PARTE: VISUALIZACIÓN DE DATOS

Esta sección cubre el fichero `menu_visualizacion.py`, que es un menú interactivo por terminal con varias opciones de visualización. El menú va en bucle hasta que el usuario elige salir, y cada opción lleva un `try-except` para que no se cierre todo si algo falla. Las gráficas las hacemos con matplotlib y seaborn.

3.1. RESUMEN OPCIONES DEL MENÚ

Opción	Descripción	Filtro	Fuente
1	Evolución de reviews por años	Categoría / Todas	MySQL
2	Popularidad de los artículos	Categoría / Todas	MySQL
3	Histograma por nota	Categoría / Todas / Artículo	MySQL
4	Evolución temporal de reviewes	Todas (una línea por cat.)	MySQL
5	Histograma de reviewes por usuario	Sin filtro	MySQL
6	Nube de palabras	Categoría (sin 'Todas')	MySQL + MongoDB
7	Nota media por categoría (extra)	Sin filtro	MySQL
8	Recomendación de artículos	Usuario + Categoría	MySQL
9	Salir	-	-

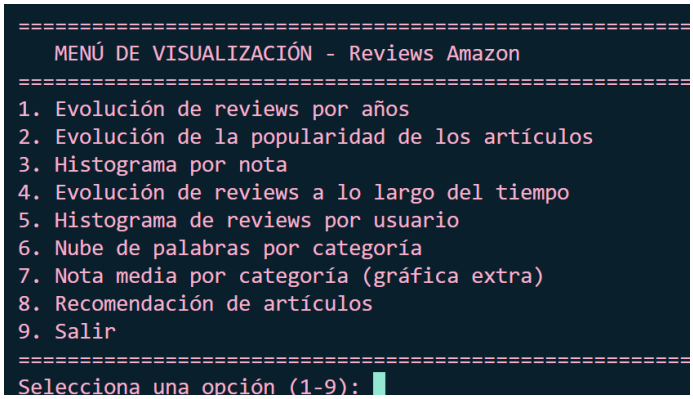


Figura 0. Despliegue del menú de visualización

3.2. DESCRIPCIÓN DE CADA VISUALIZACIÓN

OPCIÓN 1: EVOLUCIÓN DE REVIEWS POR AÑOS

Gráfico de barras con el número de reviews por año. El usuario elige una categoría o "Todas". Los datos salen de MySQL agrupando por `YEAR(reviewDate)` y, si se filtra por categoría, se hace un `INNER JOIN` con la tabla `products`.

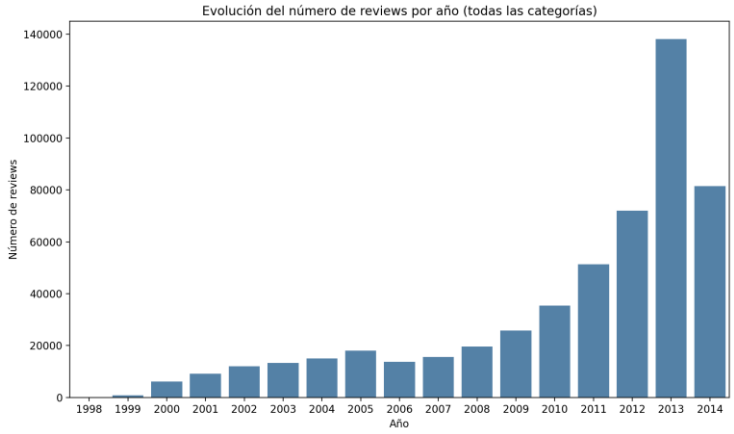


Figura 1. Opción 1 para todas las categorías

OPCIÓN 2: EVOLUCIÓN DE LA POPULARIDAD DE LOS ARTÍCULOS

Una curva con los artículos ordenados de mayor a menor popularidad (número de reviews). Se puede filtrar por categoría o verlos todos. La consulta agrupa por `asin`, cuenta las reviews y ordena de forma descendente. El eje X representa la posición del artículo en el ranking y el eje Y su número de reviews, generando la curva típica de distribución de popularidad (cola larga).

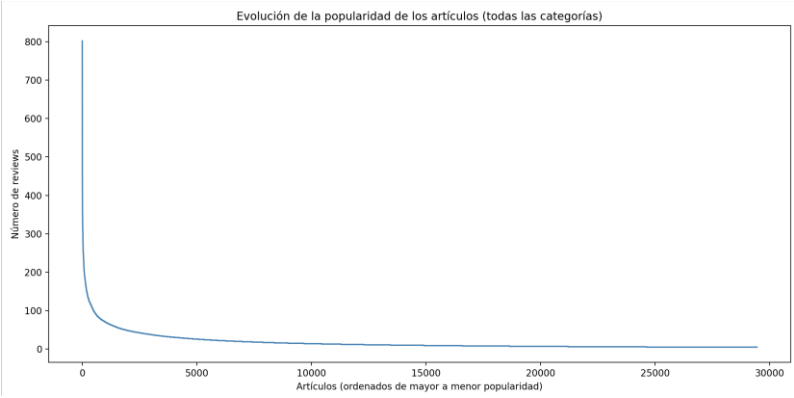


Figura 2. Opción 2 para todas las categorías

OPCIÓN 3: HISTOGRAMA POR NOTA

Histograma con el número de reviews por cada nota (1 a 5). El usuario puede elegir entre tres modos: por categoría de producto, por todas las categorías, o por un artículo individual introduciendo su ASIN. Si el ASIN introducido no existe en la base de datos, el programa lo detecta mediante una consulta previa y muestra un mensaje indicando que el artículo no existe. Cuando se filtra por categoría, se realiza un `INNER JOIN` con `products` para obtener solo las reviews de ese tipo de producto.

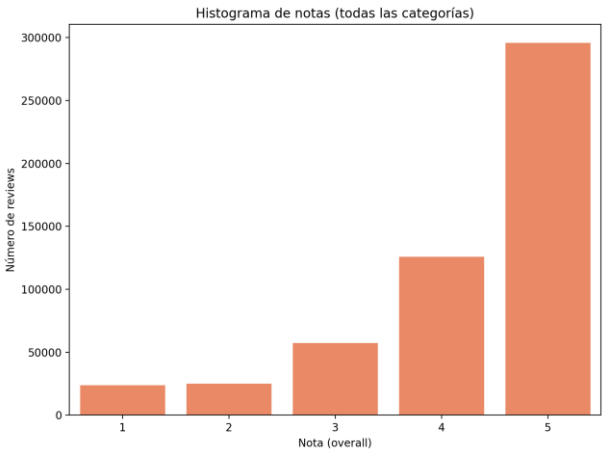


Figura 3. Opción 3 por tipo de artículo todas las categorías

OPCIÓN 4: EVOLUCIÓN DE REVIEWS A LO LARGO DEL TIEMPO

Evolución del número acumulado de reviews a lo largo del tiempo para todas las categorías simultáneamente, con una línea por cada categoría. Se utiliza el campo `unixReviewTime` (timestamp numérico) como eje X, tal como sugiere el enunciado. Para cada categoría, sacamos todos los timestamps ordenados cronológicamente y calculamos el acumulado (1, 2, 3, ..., N), representando cuántas reviews se habían publicado hasta ese momento. Utilizamos `plt.plot()` con `plt.legend()` para diferenciar las curvas.

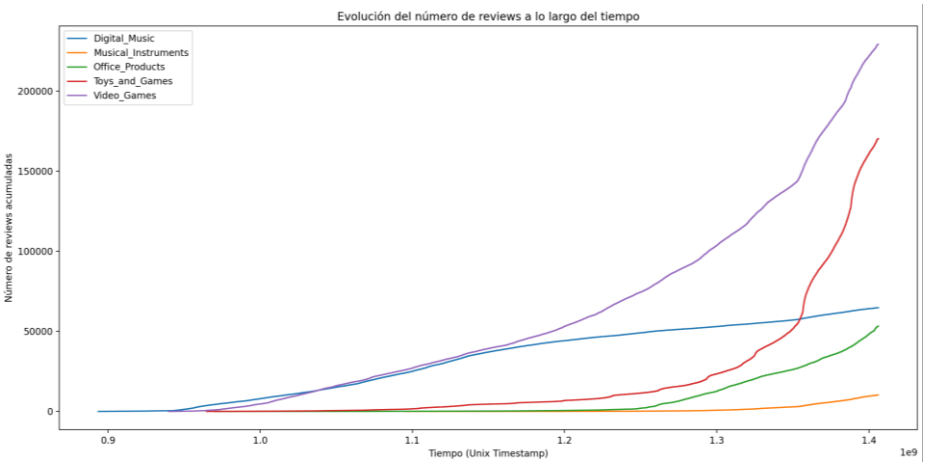


Figura 4. Opción 4

OPCIÓN 5: HISTOGRAMA DE REVIEWS POR USUARIO

Histograma de distribución donde el eje X indica el número de reviews y el eje Y indica cuántos usuarios han realizado exactamente esa cantidad de reviews. No se distingue por tipo de producto. La implementación consiste en dos casos: primero, una consulta SQL que agrupa por `reviewerID` y cuenta las reviews de cada usuario. Segundo, un conteo en Python (mediante un diccionario) de cuántos usuarios comparten cada número de reviews. El resultado muestra la distribución típica donde la mayoría de usuarios tienen pocas reviews y unos pocos tienen muchas.

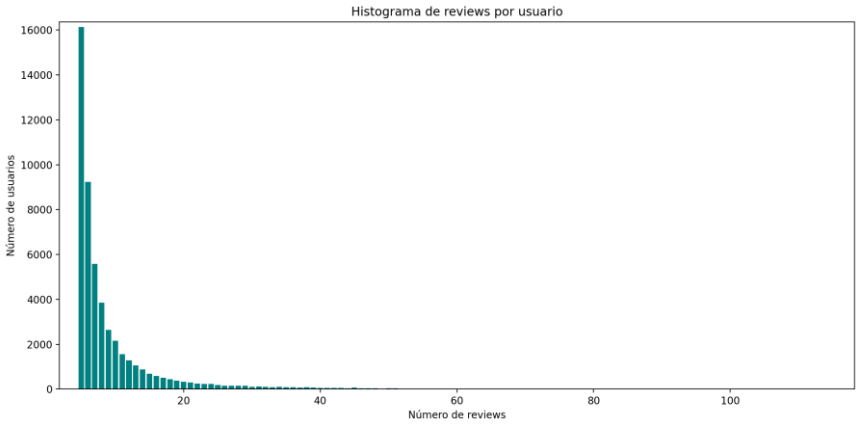


Figura 5. Opción 5

OPCIÓN 6: NUBE DE PALABRAS POR CATEGORÍA

Nube de palabras a partir del campo `summary` de las reviews de una categoría seleccionada. Siguiendo el enunciado, no permite obtener la nube para todas las categorías a la vez. Solo se tienen en cuenta las palabras de longitud mayor que 3 caracteres para eliminar conectores y palabras cortas. Esta es la única visualización que combina ambas bases de datos: los `asin` de la categoría se obtienen de MySQL (tabla `products`) y los textos summary se consultan en MongoDB usando el operador `$in`. Se utiliza la librería `WordCloud` para generar la imagen.



Figura 6. Opción 6 Musical_Instruments

OPCIÓN 7: NOTA MEDIA POR CATEGORÍA (GRÁFICA EXTRA)

Esta es la gráfica libre que hemos elegido, diferente a todas las anteriores ya que muestra la media de las puntuaciones (overall) por categoría de producto en lugar de conteos. Realizamos un `INNER JOIN` entre `reviews` y `products` agrupando por `category` y calculando `AVG(overall)`. El valor de la media se muestra encima de cada barra para facilitar la lectura.

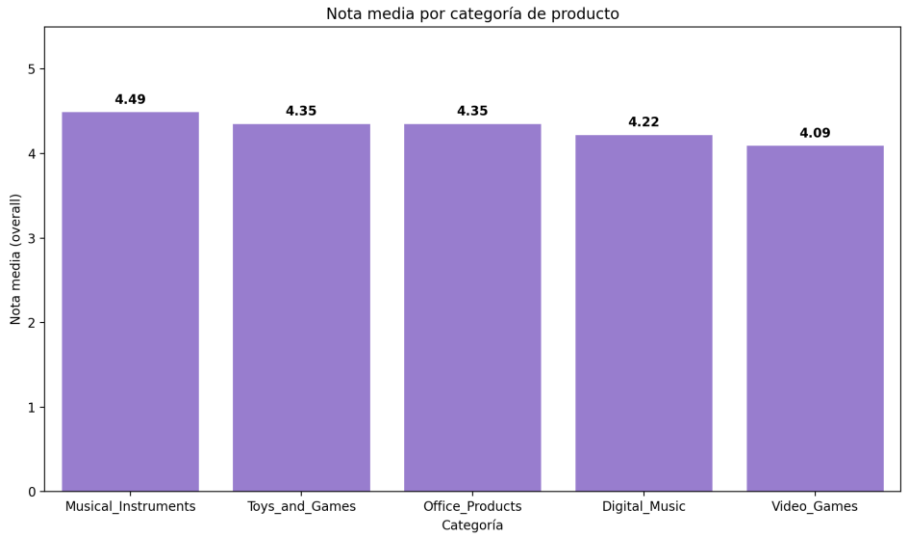


Figura 7. Opción 7

OPCIÓN 8: RECOMENDACIÓN DE ARTÍCULOS (PARTE 5)

Esta opción es la que corresponde a la quinta parte del enunciado. Dado un ID de usuario y un tipo de artículo, devuelve los 10 artículos más populares de esa categoría que el usuario no ha consumido. Primero se comprobamos que el usuario existe en la base de datos. Luego lanzamos una consulta SQL que selecciona los artículos de la categoría indicada, excluye los que el usuario ya ha puntuado mediante una subconsulta `NOT IN`, agrupa por `asin`, ordena por número de reviews descendente y limita a 10 resultados.

```
Top 10 artículos de Video_Games recomendados para A3OXHLG6DIBRW8:
------------------------------------------------
1. B00BGA9WK2 (802 reviews)
2. B0015AARJI (652 reviews)
3. B007CM0K86 (517 reviews)
4. B002VBWIP6 (465 reviews)
5. B0050SYX8W (441 reviews)
6. B000B9RI14 (425 reviews)
7. B00178630A (379 reviews)
8. B007XVTR3K (365 reviews)
9. B004EQCCI4 (355 reviews)
10. B00503E8S2 (344 reviews)
```

Figura 8. Ejemplo opción 8

3.3. DECISIONES DE DISEÑO Y DESAFÍOS

- **Función auxiliar reutilizable**: Creamos la función `pedir_categoria()` con el parámetro `permitir_todo` para centralizar la selección de categoría y evitar duplicar código. La nube de palabras la invoca con `permitir_todo=False` mientras que el resto de opciones usan `permitir_todo=True`.
- **Combinación de MySQL y MongoDB**: La nube de palabras es el ejemplo más claro de interacción entre ambas bases de datos. Se obtienen los `asin` de una categoría desde MySQL y luego se consultan los textos `summary` en MongoDB filtrando por esos `asin` con el operador `$in`.
- **Validación de entradas**: Todas las opciones validan la entrada del usuario. En el histograma por nota, si el usuario introduce un `ASIN` que no existe, se detecta mediante una consulta previa a la tabla `products` y se muestra un mensaje claro. Las opciones inválidas del menú también se gestionan sin que el programa termine.
- **Robustez frente a errores**: El bucle principal del menú envuelve cada opción en un bloque `try-except` que captura cualquier excepción inesperada (por ejemplo, un fallo de conexión a la base de datos), muestra el error al usuario y permite continuar usando el menú sin que el programa se cierre de forma abrupta.
- **Librerías de visualización**: Usamos matplotlib y seaborn. Seaborn para los gráficos de barras (`sns.barplot`) por su estética, mientras que matplotlib lo usamos directamente para las curvas (`plt.plot`) y los histogramas (`plt.bar`). La nube de palabras se genera con la librería WordCloud.

4. TERCERA PARTE: APLICACIÓN PYTHON CON NEO4J

Este apartado cubre el fichero `neo4JProyecto.py`, que tiene un menú interactivo con cuatro funcionalidades que combinan MySQL (como fuente de datos) y Neo4J (para visualizar relaciones en forma de grafo). Como en el menú de visualización, va en bucle y cada opción está protegida con `try-except`.

4.1. RESUMEN DE OPCIONES DEL MENÚ

Opción	Descripción	Entrada del usuario	Limpia Neo4J
1	Similitudes entre usuarios (Pearson)	Ninguna (usa constante `N_USUARIOS=30`)	Sí
2	Enlaces entre usuarios y artículos aleatorios	Categoría + nº artículos	Sí
3	Usuarios con más de un tipo de artículo	Ninguna	Sí
4	Artículos populares y artículos en común	Ninguna	Sí
5	Salir	-	-

4.2. OPCIÓN 1: SIMILITUDES ENTRE USUARIOS (CORRELACIÓN DE PEARSON)

Esta opción ejecuta el siguiente proceso:

OBTENCIÓN DE LOS TOP USUARIOS

Sacamos de MySQL los 30 usuarios con más reviews. El 30 está definido como constante `N_USUARIOS = 30` al principio del fichero, así que para cambiarlo solo hay que tocar esa línea, como pide el enunciado.

OBTENCIÓN DE REVIEWS

Para cada uno de los 30 usuarios obtenemos todas sus reviews (pares `asin: overal`) desde MySQL, almacenándolas en un diccionario `{reviewerID: {asin: overall}}`.

CÁLCULO DE LA CORRELACIÓN DE PEARSON

Recorremos todos los pares de usuarios (sin repetir) y calculamos la correlación de Pearson con la fórmula del enunciado. Solo miramos los artículos que ambos han puntuado. Si no tienen ningún artículo en común, no guardamos nada. Si el denominador sale cero (porque uno de los dos ha puntuado todo igual), devolvemos `None`. La función `calcular_pearson()` implementa la

fórmula completa: medias sobre los artículos comunes, numerador como suma de productos de desviaciones, y denominador como producto de las raíces de las sumas de desviaciones al cuadrado.

ALMACENAMIENTO EN FICHERO AUXILIAR

Las similitudes se guardan en un fichero CSV (`similitudes.csv`) con columnas `usuario1`, `usuario2`, `pearson`, como sugiere el enunciado.

CARGAR EN NEO4J

Creamos nodos de tipo `USER` (uno por usuario) y relaciones de tipo `SIMILITUD` con la propiedad `pearson`. La similitud es bidireccional: se crean dos aristas dirigidas (una en cada sentido) para cada par.

USUARIO CON MÁS VECINOS

Mediante una consulta Cypher que cuenta las relaciones `SIMILITUD` salientes de cada nodo y ordena descendentemente, mostramos por pantalla el usuario con más vecinos.

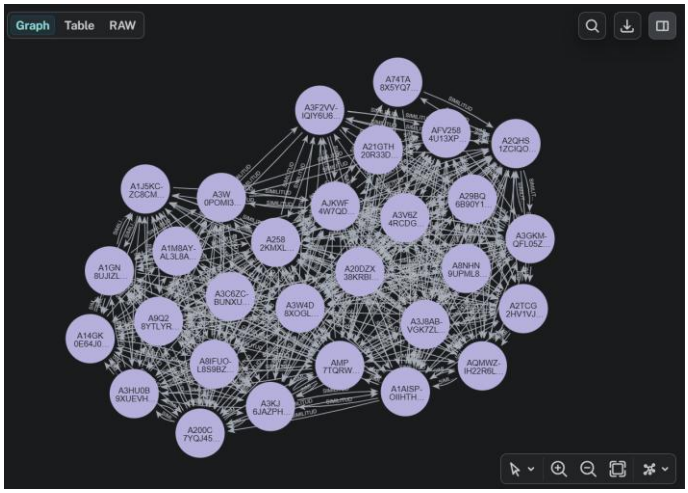


Figura 1. Opción 1

4.3. OPCIÓN 2: ENLACES ENTRE USUARIOS Y ARTÍCULOS ALEATORIOS

Se pide al usuario que seleccione un tipo de artículo (Toys_and_Games, Video_Games, etc.) y cuántos artículos aleatorios quiere seleccionar. El proceso:

- Sacamos todos los `asin` de esa categoría desde MySQL y seleccionamos `n` aleatorios con `random.sample()`.
- Limpiamos Neo4J previamente para evitar conflictos con datos anteriores.
- Creamos nodos `ARTICULO` (con propiedades `asin` y `categoria`) y nodos `USER` (con `MERGE` para evitar duplicados cuando un usuario ha puntuado varios artículos).
- Creamos relaciones `HA_PUNTUADO` con las propiedades `nota` (overall) y `tiempo` (`unixReviewTime`), cumpliendo el requisito de incluir la nota y el momento temporal de la review.
- Mostramos un mensaje al finalizar indicando que se puede consultar Neo4J.

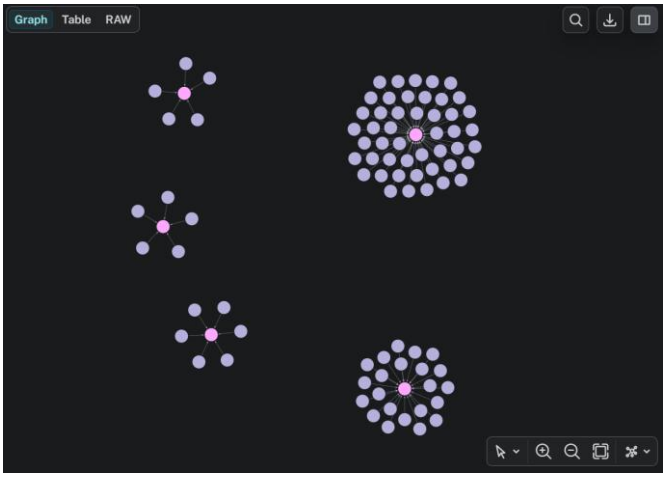


Figura 2. Opción 2 para 5 artículos de Toys_and_Games

4.4. OPCIÓN 3: USUARIOS CON MÁS DE UN TIPO DE ARTÍCULO

- Obtenemos los primeros 400 usuarios ordenados por nombre (`ORDER BY reviewerName ASC LIMIT 400`).
- Para cada uno de esos 400 usuarios, consultamos en MySQL qué categorías de artículos ha puntuado y cuántos artículos de cada tipo, mediante un `JOIN` entre `reviews` y `products` agrupando por `category`.
- Solo incluimos en Neo4J los usuarios que han puntuado artículos de más de un tipo distinto (es decir, `len(categorias_usuario) > 1`).
- Creamos nodos `USER` y nodos `TIPO_ARTICULO` (representando los tipos, no artículos individuales). Los nodos de tipo se crean con `MERGE` para evitar duplicados.
- Las relaciones `HA_CONSUMIDO` tienen la propiedad `num_articulos` que indica cuántos artículos de ese tipo ha consumido cada usuario.

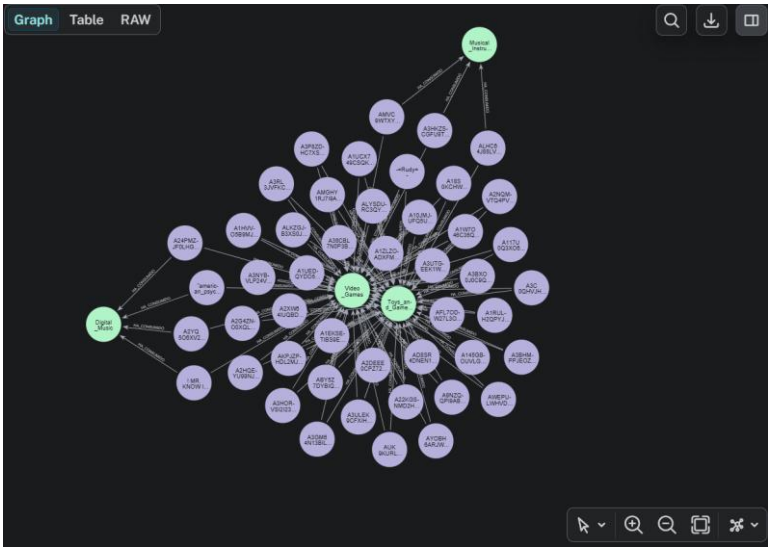


Figura 3. Opción 3

4.5. OPCIÓN 4: ARTÍCULOS POPULARES Y ARTÍCULOS EN COMÚN

- Obtenemos los 5 artículos más populares que tengan menos de 40 reviews, mediante `HAVING COUNT(*) < 40 ORDER BY total DESC LIMIT 5`.
- Creamos nodos `ARTICULO` con las propiedades `asin` y `num_reviews`.
- Identificamos todos los usuarios que han puntuado alguno de esos 5 artículos y se crean como nodos `USER`.
- Creamos relaciones `HA_PUNTUADO` entre cada usuario y los artículos que ha puntuado, con la propiedad nota.
- Calculamos los artículos en común entre cada par de usuarios: para cada par se calcula la intersección de los artículos (de los 5 seleccionados) que ambos han puntuado. Si tienen alguno en común, se crea una relación `ARTICULOS_EN_COMUN` con la propiedad `cantidad`.

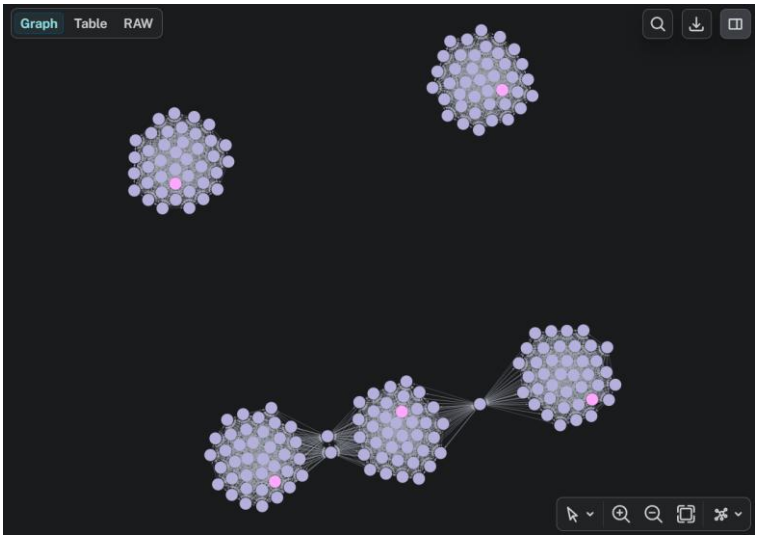


Figura 4. Opción 4

4.6. DECISIONES DE DISEÑO Y DESAFÍOS

1. **Limpieza de Neo4J antes de cada opción.** Todas las opciones ejecutan `MATCH (n) DETACH DELETE n` al arrancar para partir de cero y que no se mezclen datos entre opciones, como pide el enunciado.
2. **Constante configurable.** El número de usuarios para la opción 1 se define como constante global `N_USUARIOS = 30`, cumpliendo el requisito de que sea modificable con el mínimo cambio posible.
3. **Uso de MERGE vs CREATE.** Usamos `CREATE` para nodos que sabemos que son únicos (los usuarios de la opción 1 tras limpiar) y `MERGE` cuando un mismo nodo podría crearse varias veces (por ejemplo, un usuario que ha puntuado varios artículos en la opción 2, o los nodos de tipo de artículo en la opción 3).
4. **Eficiencia en el cálculo de artículos en común.** En la opción 4, en lugar de hacer una consulta SQL por cada par de usuarios, obtenemos antes los artículos de cada usuario en un diccionario de sets y calculamos la intersección en Python con el operador `&`, lo que es mucho más eficiente.

5. **Robustez.** El `main()` envuelve cada opción en `try-except` para que un error de conexión o de datos no termine el programa. Al cerramos correctamente el driver de Neo4J con `driver.close()`.

5. CUARTA PARTE: INSERCIÓN DE NUEVO DATASET

Aquí explixamos el desarrollo del fichero `inserta_dataset.py`, que implementa un mecanismo para insertar un quinto conjunto de datos en las bases de datos ya existentes. Hemos seleccionado el dataset Office Products 5-core (`Office_Products_5.json`).

5.1. DATASET SELECCIONADO

El fichero elegido es `Office_Products_5.json`, que corresponde a reviews de productos de oficina. Le hemos asignado la categoría `Office_Products` para diferenciarlo de los cuatro datasets originales. Tanto la ruta del fichero como el nombre de la categoría están como variables al principio del script, por si se quisiera cambiar a otro dataset.

5.2. PROCESO DE INSERCIÓN

El proceso sigue la misma lógica que `load_data.py`, pero adaptado al hecho de que las bases de datos ya contienen datos previos. Los pasos son:

- **Carga del estado actual.** Antes de empezar a insertar, consultamos qué reviewers y products ya existen en MySQL y los cargamos en dos sets(`reviewers_existentes` y `products_existentes`). Así ya sabemos de antemano qué hay ya en la base de datos y no intentamos insertar cosas que ya están, respetando que un mismo usuario puede tener reviews en varias categorías.
- **Lectura línea a línea.** Al igual que en `load_data.py`, el fichero se lee línea a línea con `for linea in fichero` y se parsea cada línea individualmente con `json.loads()`, cumpliendo la restricción de no cargar el fichero completo en memoria.
- **Inserción en MySQL.** Para cada review, miramos si el reviewer ya existe en el set; si no existe, lo insertamos en la tabla `reviewers` y lo añadimos al set. Lo mismo ocurre con los productos: si el `asin` no está en `products_existentes`, lo insertamos con la categoría `Office_Products`. La review se inserta siempre en la tabla `reviews` con la fecha parseada a tipo `DATE`. Capturamos `IntegrityError` como medida de seguridad adicional.
- **Inserción en MongoDB.** Simultáneamente insertamos un documento en la colección `reviews` de MongoDB con los campos textuales (`reviewText, summary`) y los campos de referencia (`reviewerID, asin, overall`), siguiendo exactamente la misma estructura que los documentos de los cuatro datasets originales.
- **Commits periódicos.** Se hace commit en MySQL cada 10.000 reviews y un commit final al terminar el fichero.

5.3. COMPATIBILIDAD CON EL RESTO DEL PROYECTO

Algo que nos parecía importante es que, después de ejecutar `inserta_dataset.py`, todo lo demás siga funcionando sin tocar nada:

- ➔ El menú de visualización detecta automáticamente la nueva categoría en las consultas SQL que agrupan por `category`, por lo que las gráficas reflejan los nuevos datos.
- ➔ La función de recomendación de artículos incluye `Office_Products` como opción seleccionable.
- ➔ Las opciones de Neo4J que consultan MySQL también trabajan con los datos ampliados sin necesidad de modificación.

5.4. RESUMEN DE LA EJECUCIÓN

Al finalizar la carga, el script muestra un resumen con el total de reviews insertadas, cuántos reviewers nuevos se han añadido (que no existían antes) y cuántos productos nuevos. Esto nos sirve para verificar que todo ha ido bien y para ver cuántos usuarios del nuevo dataset ya estaban en los datos originales.

Pilar Pérez Hernández
María Roca Ramón
(2ºB iMAT)

6. QUINTA PARTE: MÁS VISUALIZACIÓN Y RELACIÓN CON MACHINE LEARNING

6.1. VISUALIZACIÓN CON TABLEAU

Aparte de las gráficas generadas mediante Python, hemos reproducido tres de las visualizaciones del proyecto utilizando Tableau Public. Para ello hemops usado el fichero `exportar_datos_tableau.py`, que exporta los datos de MySQL a un fichero Excel (`datos_tableau.xlsx`) con las columnas necesarias (id, reviewerID, asin, overall, helpful_votes, total_votes, reviewDate, category) mediante un `INNER JOIN` entre las tablas `reviews` y `products`. Este Excel se importa directamente en Tableau Public como fuente de datos.

Las tres figuras que hemos seleccionado son:

FIGURA 1 – EVOLUCIÓN DEL NÚMERO DE REVIEWS POR AÑO

Gráfica de línea que muestra el crecimiento del número de reviews a lo largo del tiempo, desde 1998 hasta 2014. Se observa un crecimiento exponencial a partir de 2007, con un pico máximo en 2013 (cerca de 140.000 reviews) y un descenso en 2014 que se explica porque el dataset no recoge el año completo. Esta gráfica corresponde a la opción 1 del menú de visualización.

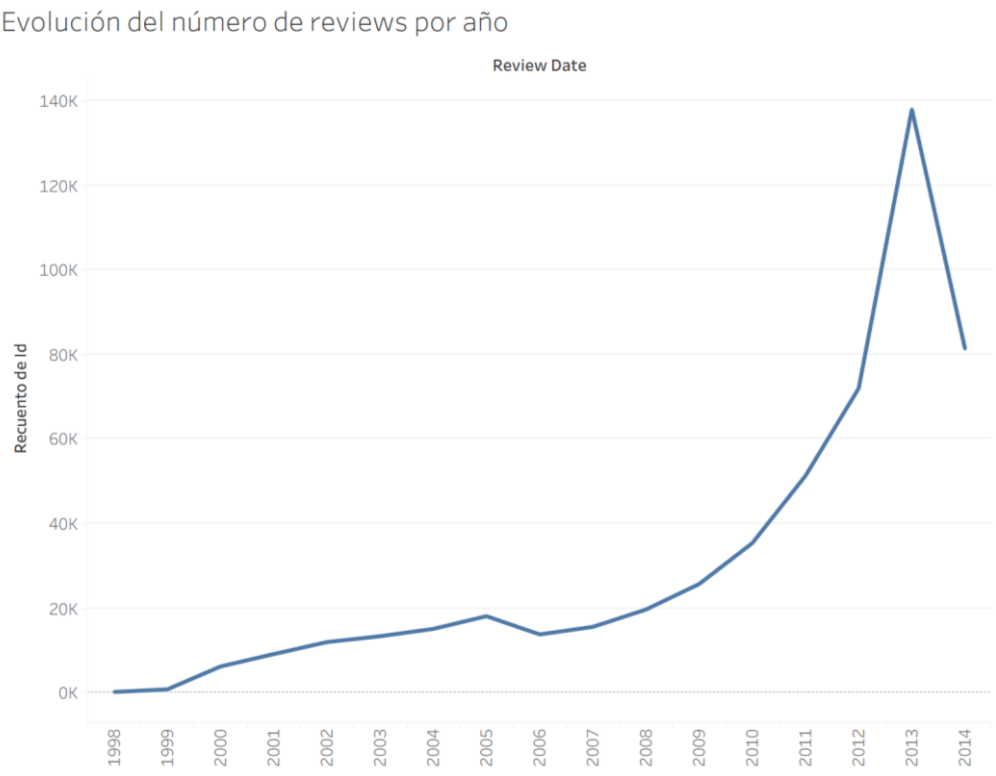


Figura 1. Evolución del número de reviews por año

FIGURA 2 – HISTOGRAMA DE NOTAS

Gráfica de línea que muestra la distribución de reviews por nota (overall). Se aprecia claramente que la nota 5 es la más frecuente (cerca de 300.000 reviews), seguida de la nota 4, y que las notas bajas (1 y 2) son las menos habituales. Esto refleja el sesgo positivo típico de las reviews online. Corresponde a la opción 3 del menú.

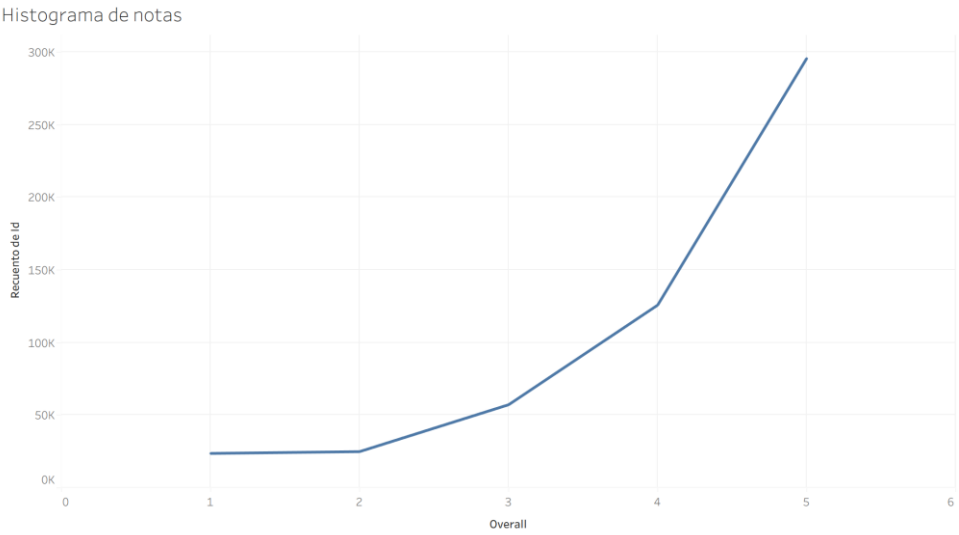


Figura 2. Histograma de notas

FIGURA 3 – NOTA MEDIA POR CATEGORÍA DE PRODUCTO

Gráfico de barras que muestra el promedio de la nota overall por cada categoría. Musical Instruments obtiene la media más alta (4,50), seguida de Toys and Games y Office Products (ambas alrededor de 4,35), Digital Music (4,24) y Video Games con la media más baja (4,09). Corresponde a la opción 7 del menú (gráfica extra).

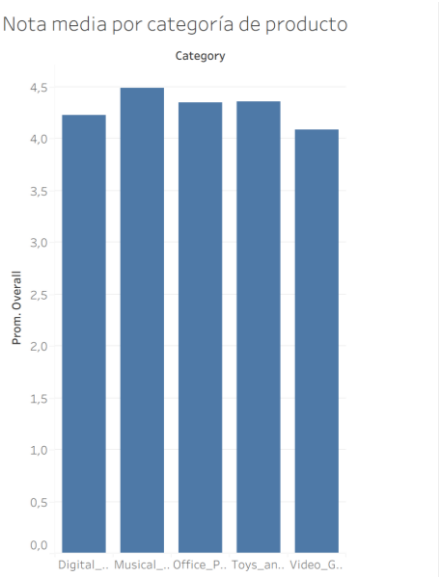


Figura 3. Nota media por categoría de producto

6.2. DESCRIPCIÓN DEL MODELO DE MACHINE LEARNING

Para el sistema de recomendación elegimos un enfroque filtrado colaborativo basado en usuarios, aprovechando que ya teníamos el cálculo de Pearson hecho en la parte de Neo4J.

El proceso tendría estas etapas:

Etapa 1 — Cálculo de similitudes. Se calcula la correlación de Pearson entre el usuario objetivo y el resto, usando solo los artículos puntuados en común. Este paso ya lo teníamos implementado en `neo4JProyecto.py`.

Etapa 2 — Selección de vecinos. Se seleccionan los K usuarios más similares al usuario objetivo (por ejemplo, K=20) que tengan una correlación de Pearson positiva. Estos son los "vecinos" cuyas preferencias se utilizarán para predecir.

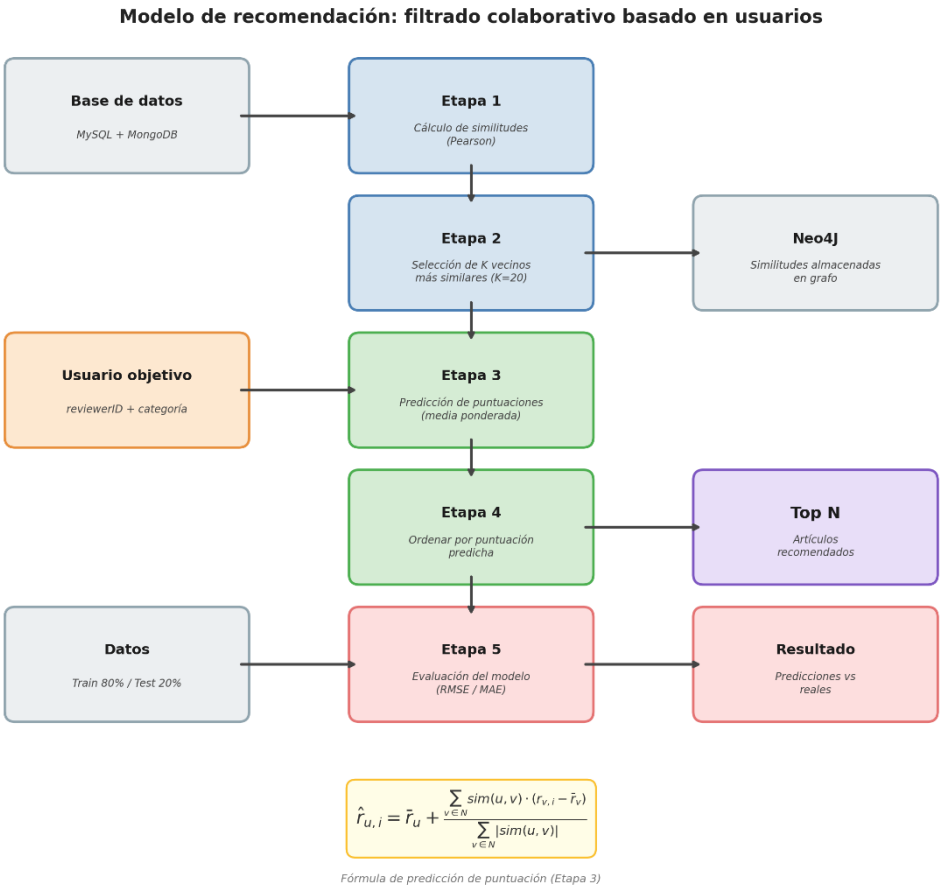
Etapa 3 — Predicción de puntuaciones. Para cada artículo que el usuario objetivo no ha consumido, estimamos que nota que le daría haciendo una media ponderada de las notas de sus vecinos, donde los pesos son las similitudes de Pearson. La fórmula sería:

```
predicción(u, i) = media(u) + Σ[sim(u,v) × (r(v,i) − media(v))] / Σ|sim(u,v)|
```

donde la suma recorre los vecinos v que han puntuado el artículo i.

Etapa 4 — Generación de recomendaciones. Se ordenan los artículos no consumidos por su puntuación predicha de mayor a menor y se recomiendan los N primeros.

Etapa 5 — Evaluación. Se dividen los datos en conjuntos de entrenamiento (80%) y test (20%). Se predicen las puntuaciones del test y se evalúa el modelo con métricas como el RMSE (Root Mean Squared Error) o el MAE (Mean Absolute Error), comparando las puntuaciones predichas con las reales.



6.3. RECOMENDACIÓN DE ARTÍCULOS POPULARES NO CONSUMIDOS

Esto lo implementamos en la opción 8 del menú de visualización (`recomendar_articulos()`). Dado un ID de usuario y un tipo de artículo, devuelve los 10 artícullos más populares de esa categoría que el usuario no ha consumido. La popularidad se mide por número total de reviews. La consulta usa `NOT IN` para quitar los artículos que el usuario ya ha puntuado, agrupa por `asin`, ordena por reviews y limita a 10.

7. SEXTA PARTE (OPCIONAL): IMPLEMENTACIÓN DEL MODELO DE MACHINE LEARNING

Además de describir el modelo en la sección anterior, lo hemos implementado en un fichero aparte llamado modelo_ml.py. Este fichero ejecuta todo el pipeline completo: obtener datos, dividir en train/test, calcular similitudes, predecir puntuaciones, evaluear con métricas y mostrar recomendaciones de ejemplo.

PARÁMETROS DEL MODELO

Los parámetros del modelo se definen como constantes al inicio del dichero, facilitando su modificación:

Parámetro	Valor	Descripción
K_VECINOS	20	Número de vecinos más similares a la predicción
PROPORCION_TRAIN	0.8	Proporción de datos para entrenamiento (80%)
MIN_ARTICULOS_COMUN	2	Mínimo de artículos en común para calcular similitud
CATEGORIA	Video_Games	Categoría sobre la que se entrena el modelo
N_USUARIOS_MODELO	200	Número de usuarios más activos considerados

PIPELINE DE EJECUCIÓN

La ejecución del modelo sigue los siguientes pasos:

1. **Obtención de datos.** Sacamos de MySQL los 200 usuarios con más reviews en la categoría Video_Games, quedándonos solo con los que tienen al menos 5 reviews (para que haya suficientes datos parala división train/test). En total nos salieron 19549 reviews.
2. **División train/test.** Las reviews de cada usuario se reparten aleatoriamente: 80% para entrenamiento y 20% para test. Usamos una semilla fija (seed=42) para que los resultados sean reproducibles. Nos quedamos 15556 reviews de entrenamiento y 3993 de test.
3. **Cálculo de similitudes.** Calculamos la correlación de Pearson entre todos los pares de usuarios usando solo los datos de entrenamiento (nunca los del test, para evitar data leakge). Solo guardamos similitudes entre usuarios que tengan al menos 2 artículos en común. Salieron 3446 pares con similitud calculable.

4. **Evaluación del modelo.** Para cada review del test, predecimos la puntuación con forma de media ponderada que describimos en la sección anterior. Comparamos la nota real y calculamos RMSE y MAE.
5. **Demostración de recomendaciones.** Elegimos 3 usuarios de ejemplo y generamos sus top 10 artículos recomendados, mostrando las puntuaciones predichas.
6. **Validación de recomendaciones.** Comprobamos si los artículos que los usuarios puntuaron bien en el test (nota >= 4) aparecen entre los artículos recomendados por el modelo.

RESULTADOS OBTENIDOS

La evaluación del modelo arrojó los siguientes resultados:

Métrica	Valor
Predicciones realizadas	1.745
Predicciones no posibles	2.248
RMSE (Root Mean Squared Error)	1,1619
MAE (Mean Absolute Error)	0,8395

El MAE de 0,84 significa que, de media, las predicciones se desvían menos de 1 punto en una escala de 1 a 5. Para un modelo de filtrado colaborativo básico, es un resultado razonable.

Las 2.248 predicciones que no pudimos hacer corresponden a casos en los que ninguno de los K vecinos del usuario había puntuado ese artículo. Esto es normal en datasets tan dispersos como el de Video Games, donde hay decenas de miles de artículos distintos y la mayoría de pares usuario-artículo no tienen puntuación.

EJEMPLO DE RECOMENDACIONES GENERADAS

A continuación se muestra un ejemplo de las recomendaciones generadas para uno de los usuarios del modelo

```
--- Usuario: A3V6Z4RCDGRC44 ---
Artículos bien puntuados en test (nota >= 4):
  B002EZOQVI (nota real: 4.0) - En top 20 recomendados: NO
  B00007KQE9 (nota real: 5.0) - En top 20 recomendados: NO
  B000E0L41K (nota real: 4.0) - En top 20 recomendados: NO
Top 5 recomendaciones del modelo:
  1. B0002A59Z4 (pred: 5.0)
  2. B005THAX6K (pred: 5.0)
  3. B002I0JIQW (pred: 5.0)
  4. B000UW1YBW (pred: 5.0)
  5. B000FQ9QVI (pred: 5.0)
```

```
--- Usuario: A3V6Z4RCDGRC44 ---
Reviews en entrenamiento: 617
Nota media del usuario: 4.3
Vecinos encontrados: 20

Top 10 artículos recomendados:
  1. B0002A59Z4 (puntuación predicha: 5.0)
  2. B005THAX6K (puntuación predicha: 5.0)
  3. B002I0JIQW (puntuación predicha: 5.0)
  4. B000UW1YBW (puntuación predicha: 5.0)
  5. B000FQ9QVI (puntuación predicha: 5.0)
  6. B00008H2IW (puntuación predicha: 5.0)
  7. B002BSC4ZS (puntuación predicha: 5.0)
  8. B000U92L8A (puntuación predicha: 5.0)
  9. B00005OWBL (puntuación predicha: 5.0)
  10. B00009OY9U (puntuación predicha: 5.0)
```

ANÁLISIS DE LOS RESULTADOS

El modelo presenta funciona razonablemente bien en la predicción de puntuaciones (MAE < 1), pero tiene las limitaciones típicas del filtrado colaborativo:

- **Problema de dispersión (sparsity).** Con más de 30.000 artículos en la categoría Video_Games, la mayoría de pares usuario-artículo no tienen puntuación, lo que limita el número de predicciones posibles (solo 1.745 de 3.993 reviews de test pudieron predecirse).
- **Cold start.** Los usuarios con pocas reviews tienen pocos artículos en común con otros usuarios, lo que dificulta encontrar vecinos similares.
- **Sesgo hacia puntuaciones altas.** Muchas recomendaciones predicen una puntuación de 5.0, lo cual refleja el sesgo positivo inherente de las reviews de Amazon (la nota 5 es la más frecuente en el dataset).

Como posibles mejoras se podrían explorar: aumentar el número de usuarios del modelo, usar un enfoque item-based en lugar de user-based, incorporar técnicas de factorización matricial, o combinar el filtrado colaborativo con filtrado basado en contenido (enfoque híbrido) utilizando los textos de las reviews almacenados en MongoDB.

8. SÉPTIMA PARTE (OPCIONAL): MEJORA EN VISUALIZACIÓN – DASHBOARD INTERACTIVO CON STREAMLIT

Como mejora opcional de visualización, hemos hecho un dashboard interactivo con Streamlit que sustituye el menú por terminal por una interfaz web. Se accede desde el navegador y permite interactuar con todas las visual8izaciones de proyecto de forma mucho más cómoda.

El dashboard se implementa en el fichero `dashboard_streamlit.py` y se acompaña de un fichero `requirements.txt` con los paquetes necesarios para su ejecución, tal como indica el enunciado.

EJECUCIÓN DEL DASHBOARD

Para ejecutar el dashboard, es necesario instalar las dependencias y lanzar Streamlit desde la terminal:

```
pip install -r requirements.txt
```

```
streamlit run dashboard_streamlit.py
```

Esto abre automáticamente el navegador en `http://localhost:8501`, donde se muestra la interfaz completa del dashboard. No es necesario modificar ningún otro fichero del proyecto: el dashboard lee la configuración de conexión directamente desde configuracion.py.

TECNOLOGÍAS EMPLEADAS

Tecnología	Uso en el dashboard
`Streamlit`	Framework web para la interfaz, layout, sidebar, widgets e interactividad
`Plotly`	Gráficas interactivas (zoom, hover, descarga PNG) que sustituyen a matplotlib
`matplotlib + WordCloud`	Generación de nubes de palabras
`PyMySQL`	Consultas a la base de datos MySQL
`PyMongo`	Consultas a la base de datos MongoDB (nube de palabras)
`pandas`	Procesamiento de datos y tablas interactivas

ESTRUCTURA DEL DASHBOARD

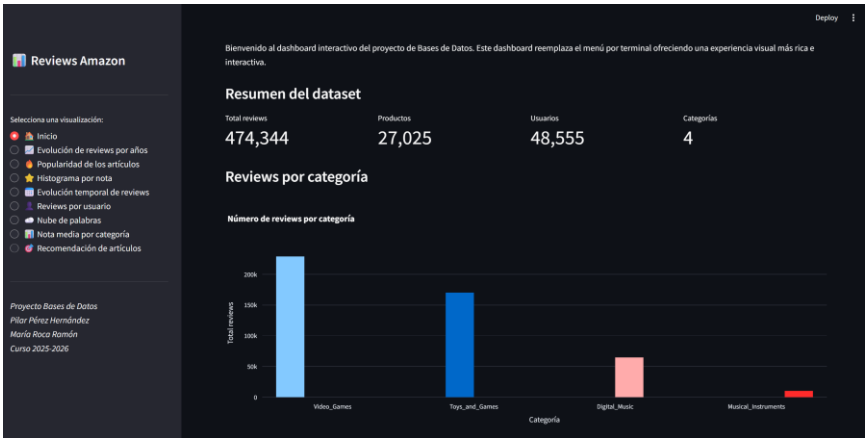
La navegación va por un sidebar lateral donde se elige la visualización que se quiere ver. Cada opción carga una página distinta con sus controles y gráficas. Las páginas son:

Página	Descripción	Equivalente menú
Inicio	Métricas generales del dataset y resumen por categoría	-
Evolución por años	Gráfico de barras interactivo con selector de categoría	Opción 1
Popularidad	Curva interactiva con zoom y estadísticas	Opción 2
Histograma por nota	Selector categoría/ASIN con validación en tiempo real	Opción 3
Evolución temporal	Líneas interactivas con fechas legibles (no timestamps)	Opción 4
Reviews por usuario	Histograma interactivo con métricas estadísticas	Opción 5
Nube de palabras	Controles de personalización (nº palabras, color fondo)	Opción 6
Nota media	Barras con escala de color y tabla detallada	Opción 7
Recomendación	Campos de entrada + tabla y gráfico de resultados	Opción 8

MEJORAS RESPECTO AL MENÚ POR TERMINAL

- **Interactividad**. Todas las gráficas están hechas con Plotly en vez de matplotlib, así que se puede hacer zoom, ver valores al pasar el raton, activar o desactivar series en la leyenda y descargar la gráfica como PNG directamente.
- **Interfaz visual**. En lugar de escribir opciones por teclado, hay selectores desplegables, campos de texto y botones. Mucho más accesible para un usuario que no sea técnico.
- **Página de inicio con métricas**. Nada máz abrir el dashboard se observa un resumen general: total de reviews, productos, usuarios y categorías. Es beneficioso para tener una visión global previa antes de meterse en las gráficas.
- **Fechas legibles**. La evolución temporal (opción 4) muestra las fechas en formato legible en lugar de timestamps Unix, aprovechando la conversión automática de pandas.
- **Tablas expandibles**. Varias visualizaciones incluyen tablas con los datos en bruto que se pueden expandir, ordenar y filtrar, añadiendo una capa de exploración de datos que el menú por terminal no ofrecía.
- **Nube de palabras personalizable.** Añadimos controles para ajustar el número máximo de palabras y el color de fondo, permitiendo al usuario experimentar con distintas configuraciones.
- **Recomendaciones visuales.** Los artículos recomendados se muestran tanto en tabla como en un gráfico de barras horizontal que permite comparar su popularidad visualmente.
- **Caché de conexiones**. Usamos el decorador @st.cache_resource de Streamlit para mantener la conexión a MongoDB abierta entre interacciones, evitando reconexiones innecesarias y mejorando el rendimiento.

CAPTURAS DEL DASHBOARD



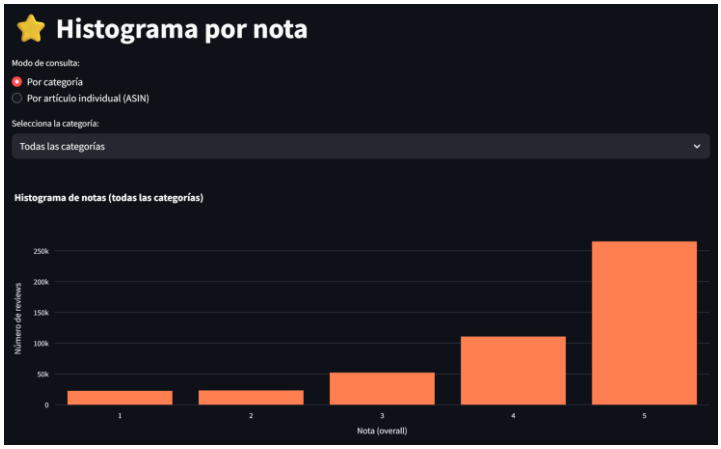
Inicio



Evolución de reviews por años para todas las categorías



Evolución de popularidad de los artículos para todas las categorías



Histograma por nota por categoría para todas las categorías



Evolución de reviews a lo largo del tiempo



Histograma de reviews por usuario





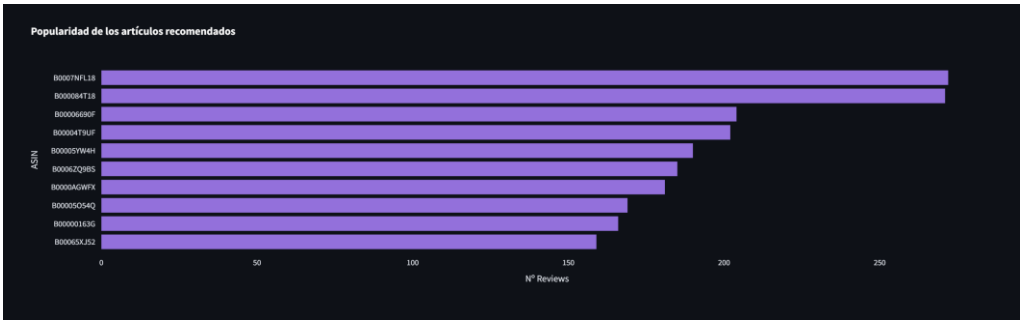
Nube palabras Musical_Instruments con fondo black y 250 palabras



Nota media por categoría de producto



Recomendación artículos Digital_Music para usuario



Popularidad de los artículos recomendados

9. CONCLUSIÓN

Este proyecto nos ha permitido trabajar de forma práctica con los tres sistemas de bases de datos que hemos visto en la asignatura. Diseñar el reparto de datos entre MySQL y MongoDB nos obligó a pensar en qué tipo de almacenamiento es más adecuado para cada tipo de información, algo que en clase se ve de forma más teórica. La parte de Neo4J nos resultó especialmente interesante al poder visualizar relaciones entre usuarios y artículos de forma tan intuitiva gracias a los grafos.

Implementar el modelo de filtrado colaborativo, fue un buen ejercicio para entender cómo funcionan los sistemas de recomendación por dentro, con sus limitaciones reales (dispersión de los datos, cold start, sesgo hacia notas altas). Y el dashboard con Streamlit le dio al proyecto una capa de presentación mucho más profesional que el menú por terminal.

En conjunto, el proyecto integra carga eficiente de datos, consultas SQL y NoSQL, análisis de grafos, Machine Learning y visualización interactiva, cubriendo un amplio abanico de tecnologías que nos serán útiles de cara al futuro.

10. DISTRIBUCIÓN DEL TRABAJO

Este proyecto ha sido desarrollado por Pilar Pérez Hernández y María Roca Ramón. La tarea la hemos distribuido entre ambas estudiantes:

Pilar Pérez Hernández ha sido responsable de:

- Diseño del esquema relacional y la justificación del reparto MySQL/MongoDB
- Implementación del fichero de carga de datos (load_data.py)
- Desarrollo completo del menú de visualización con sus 8 opciones (menu_visualizacion.py)
- Exportación de datos a Tableau (exportar_datos_tableau.py) y las tres visualizaciones en Tableau Public.
- Ejercicios opcionales: la implementación del modelo de Machine Learning de filtrado colaborativo (modelo_ml.py) y el dashboard interactivo con Streamlit (dashboard_streamlit.py), incluyendo el fichero de dependencias (requirements.txt)
- Redacción memoria del proyecto

María Roca Ramón ha sido responsable de

- Configuración del entorno de desarrollo, las credenciales de conexión y las rutas de datos (configuracion.py)
- Diseño e implementación del mecanismo de inserción del nuevo dataset Office Producs (inserta_dataset.py)
- Implementación de la aplicación con Neo4J incluyendo las cuatro funcionalidades del menú (neo4J_proyecto.py)
- Pruebas y depuración de todas las funcionalidades del proyecto (carga de datos, menú de visualización, Neo4J y dashboard)
- Validación de la integridad de los datos insertados en MySQL y MongoDB
- Revisión de la memoria del proyecto
- Elaboración del póster del proyecto

11. USO DE HERRAMIENTAS DE INTELIGENCIA ARTIFICIAL

Durante el desarrollo del proyecto hemos hecho uso de herramientas de IA generativa como apoyo, pero en ninún caso como generador de códgido final ni de ningún entregable. Todo el código ha sido diseñado, escrito y depurado por nosotras.

Concretamente, la IA se ha utilizado para los siguientes fines:

- Consulta de dudas técnicas sobre la sintaxis de consultas SQL, el uso del driver de Neo4J en Python y la configuración de PyMongo, a modo de documentación interactiva complementaria a los manuales de la asignatura.
- Apoyo en la comprensión de modelos estadísticos, como la fórmula de correlación de Pearson y las métricas de evaluación del modelo de Machine Learning (RMSE, MAE), ayudando a verificar la correcta interpretación de las fórmulas.
- Revisión y mejora de la redacción de la memoria, sugiriendo formas más claras de expresar las decisiones de diseño y los desafíos abordados.
- Diseño estético del dashboard interactivo de Streamlit, ayudando a definir la disposición visual de los elementos, la elección de emoticonos para mejorar la experiencia de usuario y la personalización de la paleta de colores y el layout general de la interfaz.
- Salida estética por terminal del menú de visualización.